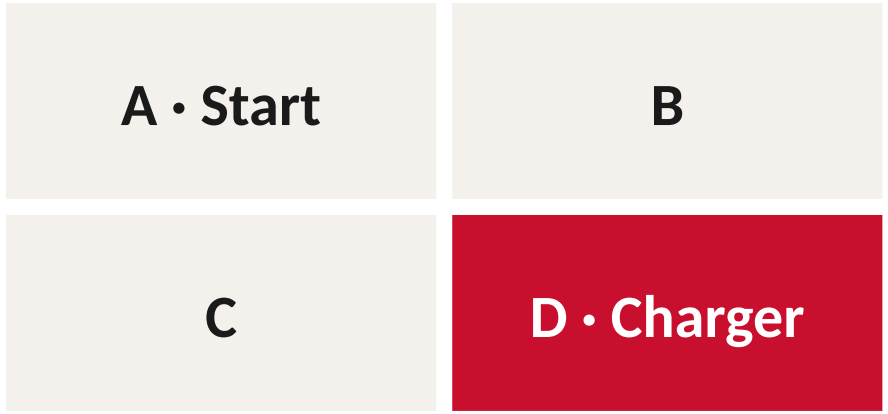
TABULAR Q-LEARNING

My robot has finished cleaning

Now it needs to find its charger so it can recharge and rest.



Which routes reach D?

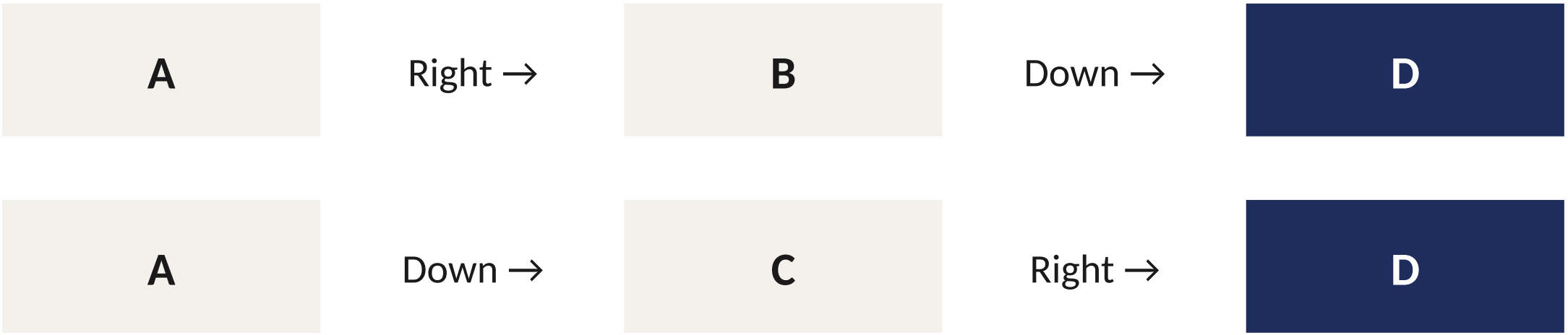


Can you find two different routes from A to the charger?

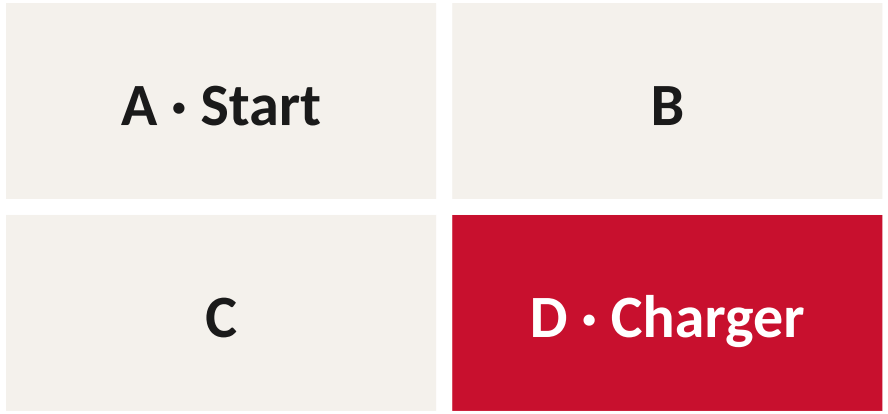
Which routes get there in the fewest moves?

Move one square at a time, horizontally or vertically.

Two routes in two moves



How can the robot learn the best moves?



Start with a Q-table: one cell for each position and move.

Update one estimate by hand.

Repeat the update in Python. This is tabular Q-learning.

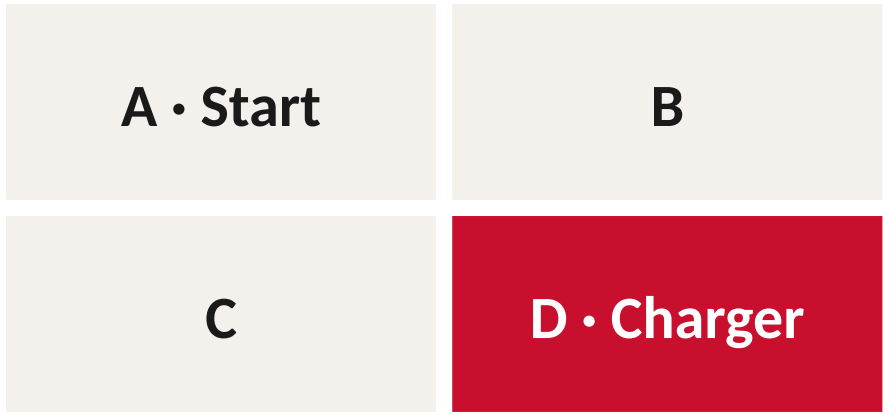
Reaching the charger earns +10

Route	First move	Reward	Second move	Reward
A → B → D	A → B	0	B → D	**+10**
A → C → D	A → C	0	C → D	**+10**

The episode ends when the robot enters D.

Which move actually earns the reward?

Can a move worth trying earn zero?



Right from A earns 0. It puts the robot one move away from +10.

What value would you give that move? How would you record it?

Two routes, same reward

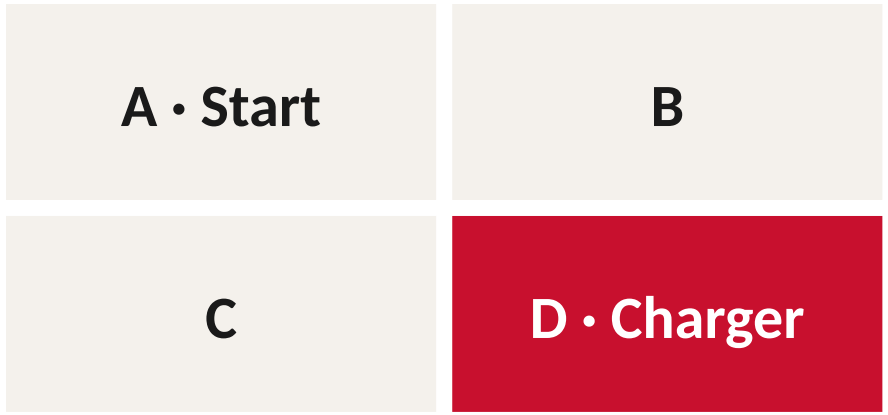
Route from B	Rewards by move	Total reward
Straight: B → D	+10	**10**
Detour: B → A → B → D	0, 0, +10	**10**

Should both routes score the same if we want the robot to charge sooner?

Use a discount factor of 0.9. Each extra move gives the future reward 90% of its previous weight.

Straight 10 **Detour** $0 + 0.9 \times 0 + 0.9^2 \times 10 = 8.1$

Which directions can we try from A?



Which directions take you to another square?

What should happen if you try to move through the wall?

Up or Left hits a wall. The robot stays at A, earns 0, and can try again.

The first Q-table

Cell	Up	Down	Left	Right
A	0	0	0	0
B	0	0	0	0
C	0	0	0	0
D	—	—	—	—

Use one row per cell and one column per direction.

Start every action estimate at 0.

The episode ends at D.

Why does the first estimate stay at zero?

The route A → B → D has a discounted return of 9 from A.

The robot takes its first step from A to B.

What does it write in Q(A, Right), and why?

The first move earns zero

Start in A. Choose Right. Arrive at B.

Immediate reward = 0. Best value in row B = 0.

Target = $0 + 0.9 \times 0 = 0$.

Q(A, Right) stays 0 on this visit.

One move updates one entry

target = reward + γ × best Q-value in the next state

new Q = old Q + α × (target − old Q)

γ = 0.9 discounts future value. α = 0.5 moves halfway.

Down from B reaches D and earns +10

Q(B, Down) = 0 + 0.5 × (10 − 0) = 5

A second trip to the charger

Cell	Up	Down	Left	Right
A	0	0	0	0
B	0	***5***	0	0
C	0	0	0	0
D	—	—	—	—

Episode 2 starts at A with the table from episode 1.

In pairs, compute the new Q-values for A/Right and B/Down.

Use $\alpha = 0.5$ and $\gamma = 0.9$.

The estimates move toward optimal Q

Cell	Up	Down	Left	Right
A	0	0	0	**2.25**
B	0	**7.5**	0	0
C	0	0	0	0
D	—	—	—	—

A to B
Target = 0 + 0.9 × 5 = 4.5
New Q = 0 + 0.5 × 4.5 = 2.25

B to D
Target = 10
New Q = 5 + 0.5 × (10 − 5) = 7.5

Three numbers for Right from A

0 Immediate reward. Also the estimate after the first try.

2.25 Updated estimate after the second A-to-B move.

9 Discounted return of the full route A → B → D.

What if we try the other route?

A · Start	B
C	D · Charger

Try Down from A, then Right from C.

Which entry changes when you reach D? What happens when you next go Down from A?

Compare the two routes by move count.

Why does the robot keep choosing Right?

Q-table after episode 2

Cell	Up	Down	Left	Right
A	0	0	0	2.25
B	0	7.5	0	0
C	0	0	0	0
D	—	—	—	—

Back at A, which move has the highest Q-value?

Does 0 mean Down is a bad move, or that we haven't tried it yet?

During training, pick a random move with probability ε.

Exploring can slow this trip, but it can reveal a good route for future trips.
The rest of the time, pick a move with the highest Q-value.

Turn the Q-update formula into Python

Episode 2 · Right from A to B

Old value 0 + 0.5 × (target 4.5 − old value 0) = 2.25

```
def update(q, state, action, reward, next_state, terminated):
    best_next = 0.0 if terminated else max(q[next_state])
    target = reward + GAMMA * best_next
    old = q[state][action]
    q[state][action] = ...
```

In pairs, write the last line using old, ALPHA and target.

Exploration after 20 episodes

Run your update in charger_grid.py. Which move would the robot pick from A?

ε = 0

Cell	Up	Down	Left	Right
A	0.00	0.00	0.00	9.00
B	0.00	10.00	0.00	0.00
C	**0.00**	**0.00**	**0.00**	**0.00**
D	—	—	—	—

ε = 0.5

Cell	Up	Down	Left	Right
A	4.37	1.94	7.08	9.00
B	8.68	10.00	7.57	8.91
C	**5.95**	**1.94**	**0.00**	**0.00**
D	—	—	—	—

ε = 0 · The robot never visited C.

ε = 0.5 · It tried Down from C. The wall leaves it in C with reward 0.
Right still shows 0 because the robot never tried Right from C.

Your turn · The robot reaches the charger

The robot moves Down from B into D and earns +10.

Old Q(B, Down) = 7.5 **Learning rate = 0.5**

1. What is the target when the episode ends at D?

2. What goes in the table? Is it the optimal Q-value yet?

A preview of Deep Q-Networks

Q-table for a 100 × 100 grid

10,000 cells × 4 moves
= 40,000 table entries

Count includes the charger, where the episode ends.

Deep Q-Network (DQN)

Given the robot's current cell, a DQN predicts four Q-values: one per move.

Up **Down** **Left** **Right**